

Deep learning: Word Embeddings

Christophe SERVAN, PhD

July 2022

1 Introduction

This tutorial aims to let you understand how to train, and use word embeddings (WE). We will use FastText to learn WE on a very short dataset which is called “Text8”.

1.1 Tools

There are two ways to install fastText, you can install it directly or use [docker](#) with the given Dockerfile.

1.1.1 Direct installation

FastText : enable you to train and use word embeddings. Installation :

- Download the installation script [FT_installation.bash](#) then type :
`bash FT_installation.bash`
WARNING : the tool will be installed by default into the “tools” directory located into your home directory.
- Once the installation is done, type
`source ~/.bashrc`
- Test your installation by typing
`fasttext --help`
the result should print the help of the `fasttext` command.

1.1.2 Using docker

You can use docker in order to create a closed environment in which you will have all the necessary tools installed. Here are the links to install under :

- Linux : <https://docs.docker.com/desktop/install/linux-install/>
- MacOS : <https://docs.docker.com/desktop/install/mac-install/>
- Windows : <https://docs.docker.com/desktop/install/windows-install/>

Once install, open a command line and then launch the following command :

```
“docker build -t myimage . ” 1
```

Then once the image have been build, you can launch a container like this :

```
“docker run -it --name containername -v hostdirectory:containerdirectory myimage /bin/bash” 2
```

The `-v` option enable to share a directory between your host machine and the container. This will enable you to edit files with your preferred script editor.

1. nb : rename the image with the name you want.

2. nb : again, feel free to rename the container with the name you want and replace the *imagenam*e by the name of the image you built)

1.2 Data

Use the wikipedia substract data file (download [here](#)) to train our word embeddings.

2 Work to do

This TP is here to help you manipulate the word embeddings (WE)

1. Fristly you will train your word embeddings
2. First task is to load the them
3. Then find the closest words
4. Finally, you will process analogy

2.1 Train word embeddings

The main idea is to train your first word embeddings model. WARNING : Read carefully the documentation available here : <https://github.com/facebookresearch/fastText>

2.1.1 Training

Train two models for each dataset, with a vector dimension of 50, 5 epochs of training and a window size of 10.

1. Train a cbow model, explain your command line
2. Train a skip-gram model, explain your command line
3. Give example of the vector file, describe it

2.2 Loading word embeddings

Loading the WE is not difficult in itself. To make computations fast, we store the whole set of embeddings in a numpy array of shape (num words, dimensions). We also build a dictionary (vocab) for mapping words to row index in this array, and another (rev vocab) for mapping indexes back to word forms.

```
def load(filename):
    vocab = {}
    rev_vocab = []
    lines = open(filename).readlines()
    vectors = np.zeros((int(lines[0].split()[0]), int(lines[0].split()[1])))
    for i, line in enumerate(lines):
        tokens = line.strip().split()
        if (i > 0) :
            vocab[tokens[0]] = i-1
            rev_vocab.append(tokens[0])
            vectors[i-1] = [float(value) for value in tokens[1:]]
    return vocab, rev_vocab, vectors
```

2.2.1 Loading

1. write in Python the necessary script to load the WE
2. what “vocab”, “rev_vocab” and “vectors” stand for ?

2.2.2 Similarity

Compute the cosine similarity for each model and each datasets

1. Explain what is the cosine similarity.
2. How to compute it in python using numpy? and using scipy?
3. What is the cosine similarity between the vectors representing “dog” and “cat”, and what about “dog” and “dentist”?
4. What is the closest word to “bank” : “river” or “trade”?

2.3 Closest words

Now you have your WE model, you will use a trick to compute all the closest words. This can be done once for all, by computing the dot product between the matrix containing all vectors and the transpose of the target word vector (`np.dot(vectors, v.T)`). Then we can use some numpy trick to recover the indices of the `n` highest scores.

```
def closest(vectors, vector, n=10):
    n=n+1
    scores = np.dot(vectors, vector.T)
    indices = np.argpartition(scores, -n)[-n:]
    indices = indices[np.argsort(scores[indices])]
    output = []
    for i in [int(x) for x in indices]:
        output.append((scores[i], i))
    return reversed(output)
```

2.3.1 Code the function

1. What pre-process to all the vectors do you NEED to do before using the dot product instead of the cosine?
2. What “argpartition” stand for?
3. Add Comments to the code.

2.3.2 Analysis

1. What are the closest words to “apple”?
2. What about other words (close to “apple”)?
3. Can you find words which have a strange neighborhood?
4. Check your answers using `scipy.spatial.distance.cosine` regarding the scores obtained what can you conclude about the dot product?

2.4 Analogy

Word analogies can be exposed by translating a word vector in a direction that corresponds to a linguistic or semantic relationship between two other words. So if we have $w1$ and $w2$ in relation $R(w1, w2)$, we can compute the relation $r = vec(w2) - vec(w1)$, and then apply this relation to the vector of another word $vec(w3)$ like this : $vec(w2) - vec(w1) = vec(w4) - vec(w3)$

Therefore, the idea is to use the closest function to find vectors similar to $vec(w4)$ where : $vec(w4) \simeq vec(w2) - vec(w1) + vec(w3)$.

2.4.1 Code the analogy function

1. Use the previous code in order to code it.

2.4.2 Solve the analogies

1. “paris”(w1) is to “france”(w2) what “delhi”(w3) is to ... (w4)?
2. “gates” - “microsoft” + “apple” = ...?
3. “king” - “man” + “woman” = ...?
4. “slow” - “slower” + “fast” = ...?

2.5 Extension

Compare your WE against one that have been already trained on very large data and available here : <https://dl.fbaipublicfiles.com/fasttext/vectors-crawl/cc.en.300.vec.gz>. Redo the previous closest words and analogy exercises.